如何开发智能体：an Example

An Example

Building an autonomous agent that can diagnose and fix network problems inside the SEED emulator is an ambitious and highly valuable research direction. Because SEED abstracts the network into distinct layers (Base, Routing, Service) and runs on containerized backends (like Docker or Kubernetes), your agent will need to bridge the gap between high-level reasoning and low-level system commands.

To get started, it is best to approach this systematically by building the agent's capabilities in phases: Perception (Read), Reasoning (Think), Action (Write), and Verification (Learn).

Here is a detailed breakdown of the tasks to develop this troubleshooting agent.

Phase 1: Environment Perception (Building the "Eyes and Ears")

Before an agent can fix a problem, it needs to see the network state and understand what the "healthy" baseline looks like.

- **Task 1.1: Topology and Ground Truth Parsing**
  - Write a module that allows the agent to ingest the SEED compilation output (the intended Base Layer and Routing Layer configurations).
  - The agent needs a structured representation (like a JSON graph) of the intended Autonomous Systems, physical links, and expected BGP/OSPF peerings.
- **Task 1.2: Execution Backend Integration**
  - Develop a secure, read-only connector between the agent and the underlying execution backend (Docker daemon or Kubernetes API).
  - The agent must be able to target specific Virtual Nodes or physical containers.
- **Task 1.3: Telemetry and Tool Tooling (The "Read" Tools)**

- Equip the agent with a suite of atomic diagnostic functions it can call.
- **L2/L3 Checks:** Functions to execute `ping`, `traceroute`, `ip addr`, and `ip route` inside specific containers.
- **Protocol Checks:** Functions to query routing daemons (e.g., parsing `birdc show protocols` to check BGP session states).
- **Resource Checks:** Functions to check CPU, memory, or interface packet drops.

Phase 2: Diagnostic Reasoning (Building the "Brain")

This is where the Large Language Model (LLM) shines. You need to implement the logic that allows the agent to map symptoms to root causes.

- **Task 2.1: Implement a ReAct (Reasoning and Acting) Loop**
  - Structure the agent's prompt to follow a ReAct framework: *Observation* (What is broken?), *Thought* (What could cause this?), *Action* (What tool should I run to test this hypothesis?).
- **Task 2.2: Prompt Engineering for Network Diagnostics**
  - Design system prompts that prime the agent with networking fundamentals. It needs to know that if `ping` fails, it should check the routing table, and if the routing table is empty, it should check the BGP daemon logs.
- **Task 2.3: Fault Injection Sandbox (For Training/Testing)**
  - Before letting the agent loose, write scripts to intentionally break a healthy SEED topology in predictable ways.
  - Examples: Bring a network interface down (`ip link set dev eth0 down`), inject a bad iptables drop rule, or introduce a typo into a BGP configuration file.

Phase 3: Action Execution (Building the "Hands")

Once the agent isolates the root cause, it must generate and safely apply the fix to the running emulator.

- **Task 3.1: Remediation Tooling (The "Write" Tools)**

    - Develop API endpoints or secure shell wrappers that allow the agent to push changes.
    - Examples: Restarting a service (`systemctl restart bird`), modifying firewall rules (`iptables -F`), or rewriting a configuration file and reloading the daemon.
- **Task 3.2: State Reversal and Safety Limits**
    - Agents can hallucinate and break things further. Implement a "checkpointing" or "undo" mechanism.
    - If the agent applies a fix and the network state worsens, it must be able to automatically roll back the container state to the moment before it intervened.
- **Task 3.3: Action Formatting**
    - Ensure the agent outputs its intended commands in a strictly parsable format (e.g., JSON) so your execution engine can sanitize the commands before running them as root inside the SEED containers.

Phase 4: Verification and Reporting (Closing the Loop)

The agent must verify its own work and explain its reasoning to the human researcher.

- **Task 4.1: Post-Fix Validation**
    - Program the agent to automatically re-run the exact diagnostic checks (from Phase 1) that originally failed. It should not mark the task as "Complete" until the `ping` succeeds or the BGP session reads `Established`.
- **Task 4.2: Incident Report Generation**
    - Have the agent summarize the entire lifecycle of the incident into a structured log: Initial Symptom → Diagnostic Steps Taken → Root Cause Identified → Actions Applied → Final State.

Where to start? I recommend starting entirely in **Phase 1**. Do not give the agent any ability to fix things yet. Build a simple "Read-Only Observer Agent." Intentionally break a link in your SEED emulator, ask the agent "Why can't Node A

reach Node B?", and focus on getting the agent to successfully use `ping` and `ip route` tools to locate the exact broken interface.